SpaceOps Conferences
28 May - 1 June 2018, Marseille, France
2018 SpaceOps Conference

10.2514/6.2018-2710

Modernizing Telemetry Displays

Ms. Sabina Hurley[1], Mr. David Jones[2] , Mr. Mark Baski[3], and Mr. Paul Viens[4]

Northrop Grumman Aerospace Systems, Cambridge, Massachusetts, 02138

[1] Chandra Flight Operations Manager, Chandra Flight Operations Team, 60 Garden St. CAMA/MS33, Cambridge, MA 02138, USA, sabina.hurley@ngc.com
[2] Chandra Ground Software Engineer, Chandra Flight Operations Team, 60 Garden St. CAMA/MS33, Cambridge, MA 02138, USA, david.a.jones@ngc.com
[3] Chandra Software Manager, Chandra Flight Operations Team, 60 Garden St. CAMA/MS33, Cambridge, MA 02138, USA, mark.baski@ngc.com
[4] Chandra Lead Systems Engineer, Chandra Flight Operations Team, 60 Garden St. CAMA/MS33, Cambridge, MA 02138, USA, paul.viens@ngc.com

Downloaded by 149.154.161.236 on April 12, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2018-2710

Advancements in telemetry-distribution systems bring unprecedented access to both real-time and historical spacecraft telemetry. With the hurdle of data-access removed, options for displaying the data in efficient and intuitive ways emerge. Building upon a telemetry back-end capability that can rapidly deliver real-time and historical telemetry through simple queries, the Chandra Flight Operations Team is developing new telemetry clients that move beyond the traditional tabular displays and time-based plots. Each of these new clients simplifies remote access to telemetry for off-hours response activities, enhances situational awareness, enables intuitive troubleshooting, and/or improves efficiency. Web-based telemetry displays provide broad flexibility, a large support community for developers, and easy access for users. Leveraging the flexibility of toolsets such as Blender and JavaScript, the Chandra X-ray Observatory Flight Operations Team has developed intuitive and interactive web-based displays that improve the ability to rapidly contextualize out-of-limits data and streamline anomaly response. An interactive, 3-D spacecraft model showing thermistor locations and temperatures, where thermistor color indicates absolute temperature or limit state, provides both an at-a-glance subsystem health check and an intuitive tool to put the data in context when a temperature does go out of limits. An interactive fault tree diagram fed with real-time telemetry guides rapid identification of the lowest-level fault when a spacecraft safing-action occurs. Telemetry dashboards that display traditional tabular data with smart features such as hover-over description, roll-up limits, and dynamic layout for mobile friendly viewing allow telemetry access from anywhere with any device. Interactive trending plots that merge historical and real-time data provide on-the-fly access to key telemetry points and rapid contextualization of out-of-limits behavior. Removing the access hurdle also opens up opportunities to add telemetry to existing tools to improve workflow and operator situational awareness. On the Chandra program, existing Mission Planning and Operations tools have been enhanced with telemetry. Overlaying telemetry on model-predicted values for angular momentum, body rates, sun angles, and temperatures enables the operator to pick up subtle behavior changes well before any limits are exceeded. Displaying live star tracker data in a star selection tool aids in understanding camera performance and identification of likely causes for dropped stars. Feeding the star data into pattern-matching algorithms can then enable near-real-time attitude determination, significantly enhancing the ability to recover from loss of attitude reference. Embedding live telemetry verification tables into full-text Standard Operating Procedures, written in Microsoft Word or HTML, eliminates inefficient back-and-forth scanning between the procedure and telemetry displays. We will work through these examples, discussing the use case, development, and implementation for each. The tools are largely based on open-source software, designed to be platform independent, and were developed by a small, dedicated team. The concepts and technologies used lend themselves to further expansion and customization for missions of every size.

Copyright © 2018 by the American Institute of Aeronautics and Astronautics, Inc.
The U.S. Government has a royalty-free license to exercise all rights under the copyright claimed herein for Governmental purposes. All other rights are reserved by the copyright owner.

Downloaded by 149.154.161.236 on April 12, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2018-2710

I. Introduction

Tabular data displays and time-based plots have been the standard for display of real-time spacecraft telemetry data for decades. Limitations in processing capability and access to the telemetry stream have long hindered moving past these standards. However, advances in both hardware and data management have now made it possible to remove both hurdles with a reasonable investment of resources. In Chandra's case, a new telemetry distribution system, dubbed MAUDE, was developed to greatly improve access to telemetry.

MAUDE is a system designed to provide easy access to Chandra telemetry data from nearly anywhere on nearly any device [1]. It was originally created to address a single, simple, goal: Provide data for plotting a single MSID over any timeframe to a user within one second. Its capabilities have since expanded with the overall purpose of quickly and easily providing secure access to telemetry data to users on their device of choice and with minimal coding required by client developers. The chosen interface for MAUDE access is HTTP (and HTTPS) due to its ubiquity and the availability of HTTP APIs in nearly all modern programming languages and platforms. Some of the key features of MAUDE include: easy access to flight, simulator, and Deep Space Network (DSN) data from nearly anywhere (with secure login), HTTP (web server) interface, real-time data distribution, quick plotting of large timeframes, and fast and simple client development on any platform. After considering performance (most important), cost, institutional skill sets and knowledge base, fit to the data set, modularity, vendor and user community support, and scalability, the Chandra team selected HDF5 for the telemetry data store and HTTP / Apache Tomcat (Java) coded with REpresentational State Transfer (REST) architectural principles for the interface that make MAUDE. Together these technologies achieved the goal of secure telemetry access from anywhere with any device and opened up a wide array of opportunities for client development.

With exciting opportunities for accessing telemetry and integrating it into new platforms, client development took off quickly. The overarching goals of client development were to simplify remote access to telemetry for off-hours response activities, enhance situational awareness, enable intuitive troubleshooting, and improve efficiency. Inspiration was taken from a variety of sources, including the Chandra science community, sports reporting, financial analysis websites, and interactive graphics from sources like the New York Times. The ubiquity of the http interface lowers barriers for development and allows the team to take advantage of a wide array of available platforms, programming languages and development tools. This has led to the rise of many customized client applications to aid real-time and back-orbit telemetry analysis and visualization. To date, clients are up and running, written in MATLAB, JavaScript, Qt, R, HTML, and Microsoft Word. The remaining sections of this paper will present the use case, development, and implementation for a sample of these telemetry clients.

II. ChandraDash - Telemetry Dashboard

Large tabular telemetry displays with color-coded limit flagging are commonplace in spacecraft operations. With tens to hundreds of mnemonics per display it can be difficult to locate any given telemetry point quickly and maintain a high-level view of the spacecraft health and safety. Enhancing traditional tabular data displays with smart features such as hover-over description, expandable subsections with roll-up limits, and dynamic layout for mobile friendly viewing make these displays more versatile and user friendly. ChandraDash implements all of these features in a dynamic telemetry web page. ChandraDash starts with a roll-up view of the major categories of telemetry the Chandra Flight Controllers access and check during a real-time support. Each category is color-coded to show the most severe alert level for the category. The operator can click on the heading to expand the section and quickly drill down to find the mnemonics that are out of state. Figure 1 shows this capability in a mobile-friendly view; it works similarly in views for larger screen sizes.

By dynamically altering the view to fit window size, ChandraDash provides simple and familiar telemetry access from any platform. It is as usable on a cell phone as it is on a large widescreen monitor. Figure 2 shows this dynamic resizing as a window size changes from a view friendly for mobile phones to a view that effectively uses the space available on a widescreen monitor.

ChandraDash is a single-page JavaScript-based web application, drawing upon the Bootstrap front-end framework for its visual elements and responsive design. HTML tables within the page are the basis for the layout of the display and the selection of telemetry points shown. Backbone.js provides the foundation for handling user interactions and coordinating changes to the display as live data flows to the application. We favored a single-page web application over an app for ease of development and use; as implemented, ChandraDash runs in almost any web browser on any platform, whereas additional complexity would be introduced with maintaining and supporting an app across multiple platforms.

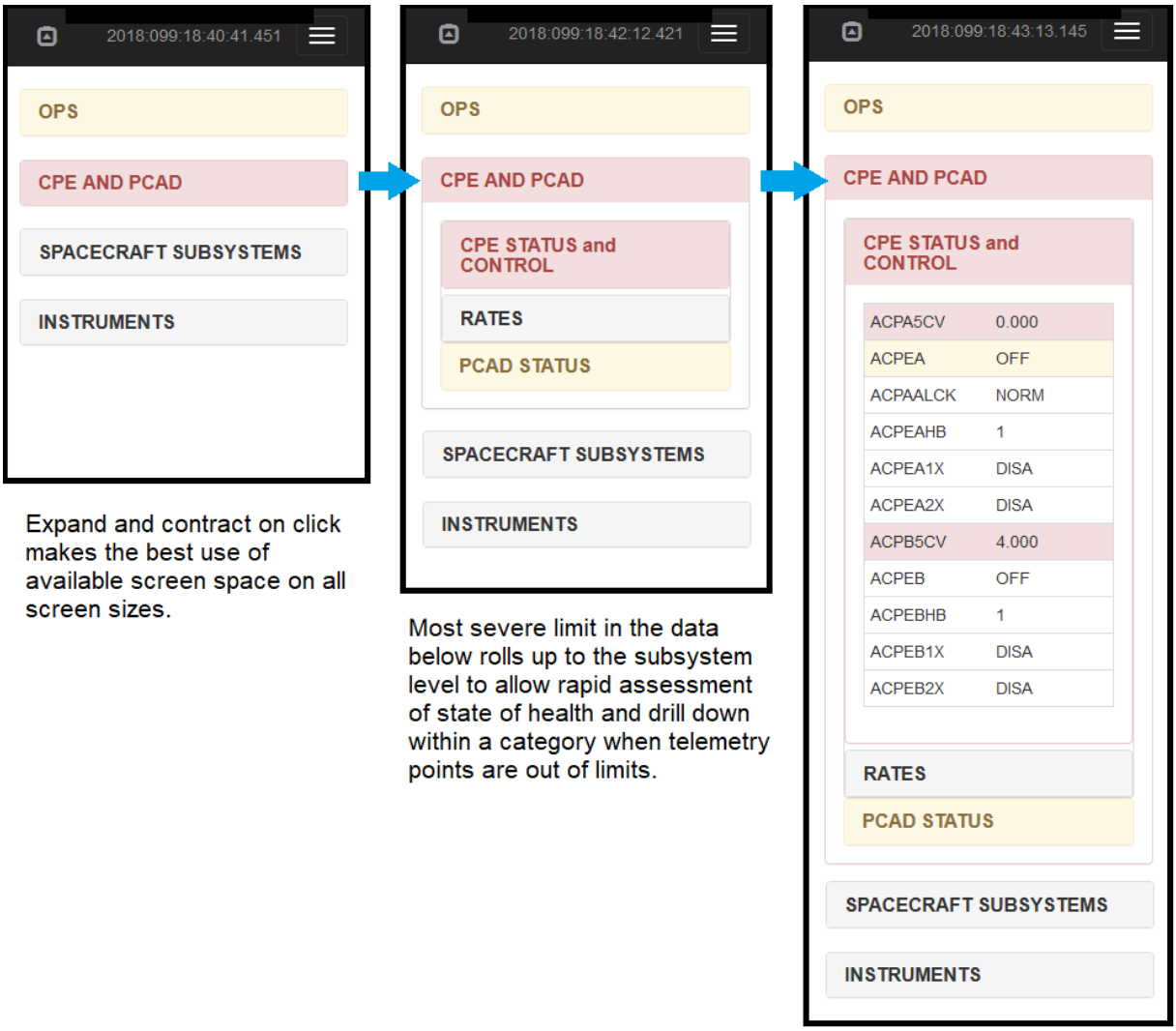

Fig. 1 Chandra Dashboard Roll-up Limits

Downloaded by 149.154.161.236 on April 12, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2018-2710

Downloaded by 149.154.161.236 on April 12, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2018-2710

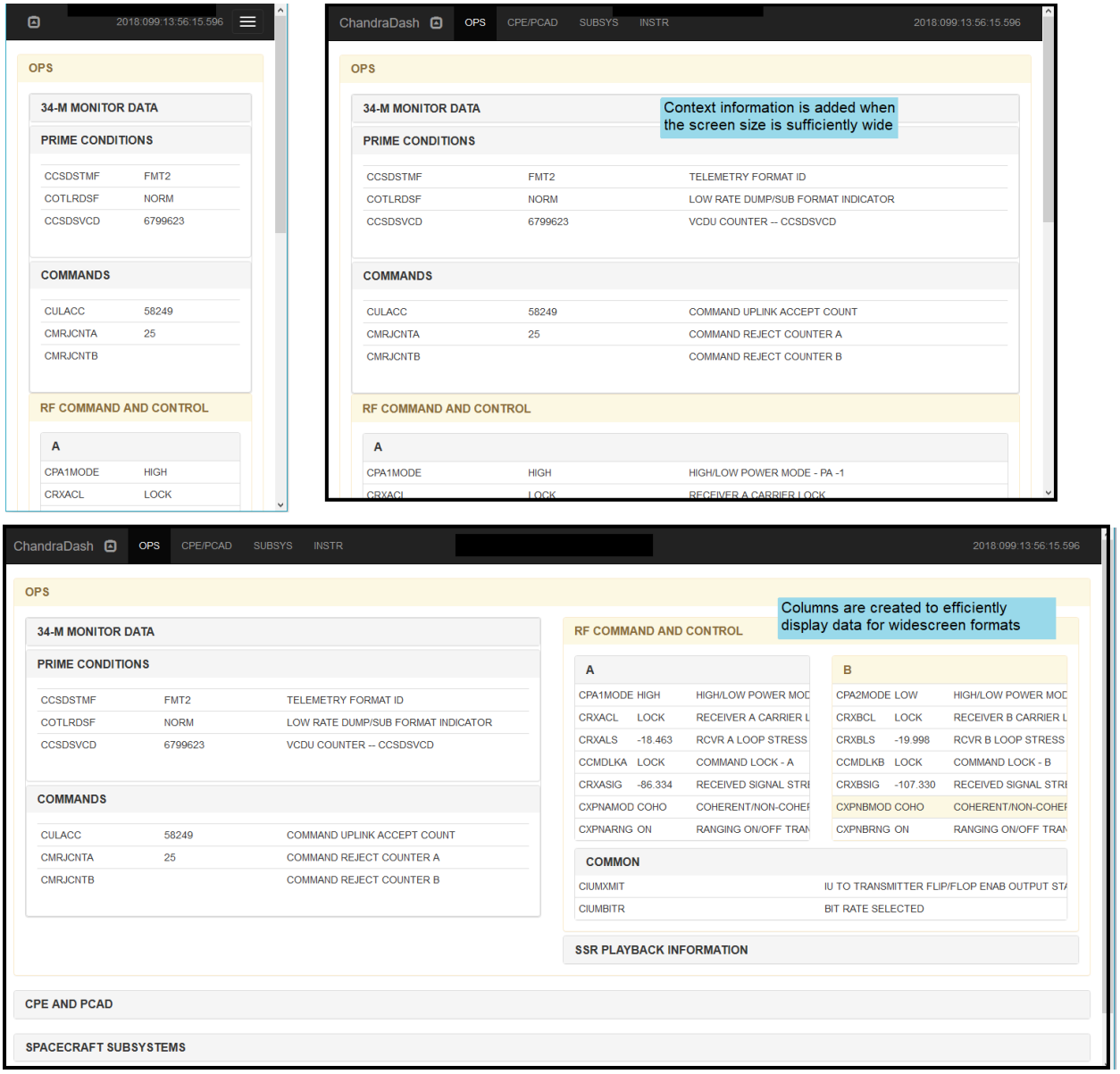

Fig. 2 Chandra Dashboard Dynamic Resizing

III. Real-Time Fault-Trees

The Chandra spacecraft is equipped with a safing subsystem that monitors for survival-threatening conditions and takes action to address them. The system is designed as a set of monitors that, when tripped, initiate predefined fault response actions, often contained in Stored Command Sequences (SCSs). In all, over eighty different monitors can trigger safing actions. Once an off-nominal configuration is detected, the flight controllers must determine what caused the reconfiguration and if it was successful. With so many possible triggers and responses, this can be a time consuming process.

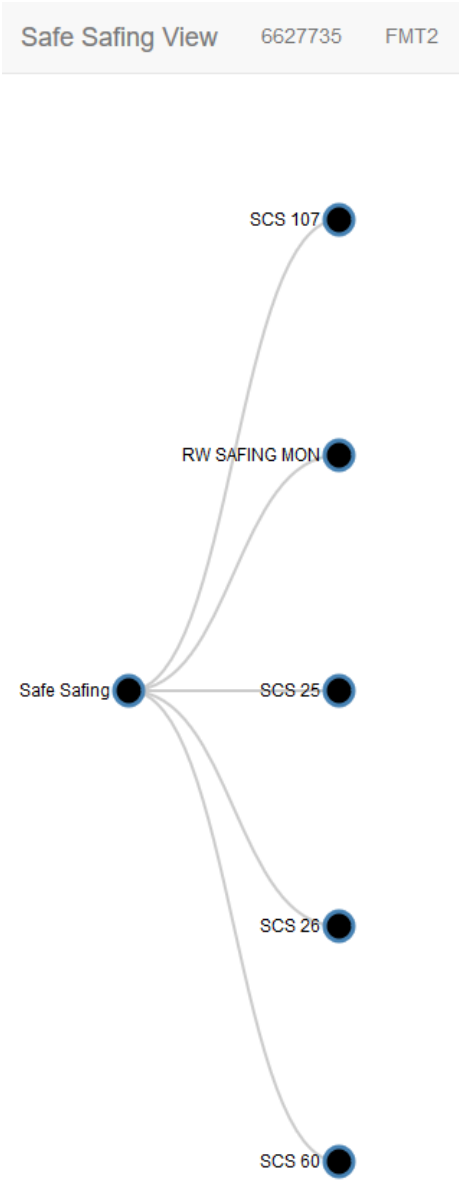

Fig. 3 Safing Fault Tree - Nominal Configuration

When a safing action occurs, the flight controllers follow a procedure that uses a telemetry key and/or the On-Board Computer (OBC) error log to identify the most likely trigger for the action. Once a trigger and safing sequence is identified, the procedure calls out steps to ensure the sequence executed properly. This process often consumes more than half the available communications time for the first support after the safing action execution, thus reducing the time available for any necessary or desirable response actions. Chandra's nominal communications support schedule calls for one 1-hour support every eight hours. In order to best use the available response window, the time to identify the cause and assess the performance of a safing action must be minimized.

In an effort to streamline analysis of safing actions and maximize the time available for response, the logic from the telemetry key used by the flight controllers was encoded into a web-based display. This enables rapid identification and intuitive display of the lowest level fault after a safing action occurs. Once the action is identified, the display switches limit sets to expect successful fault response measures and flag any discrepancies given the new expected states. The interactive display uses roll-up limits to guide the user to the identified safing action, while still allowing exploration of other possible actions. A capability to use flight software error log dump data to override the telemetry keys was also added to provide a means to force limit switching in scenarios where multiple actions are suspected or the spacecraft did not respond as expected.

Once verification is complete, this display is expected to reduce the process of identifying and verifying a safing action to a few minutes, greatly improving the time available for any response actions. Annotated examples of the real-time safing fault tree are shown in Fig. 3 to Fig. 5. Figure 3 shows the fault tree when Chandra is configured nominally and no safing actions have executed. All of the circles are black because no safing actions have executed or are suspected of executing. Figure 4 shows the fault tree after execution of a safing action. The lowest level action is shown to the far right, with the resulting higher level actions tiered to the left, in a familiar fault tree format. Operators can click on each circle for more details. A green circle indicates a suspected safing action that completed nominally, yellow designating a safing action that completed, but has a caution (yellow) limit in one of its verifiers, and red for a warning (red) limit in the verifiers. Hovering over a circle shows the indicators for each suspected safing action (i.e. why it is suspected) and the elements of a successful execution. Clicking on a circle opens or closes that node for exploration of additional actions, allowing manual override of the auto open/close behavior that makes the tree intuitive to use. For actions that can be triggered by a lower level action (e.g. Science Instrument Safing), a separate node to indicate the success of the run is provided. Figure 5 shows this feature.

Downloaded by 149.154.161.236 on April 12, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2018-2710

Downloaded by 149.154.161.236 on April 12, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2018-2710

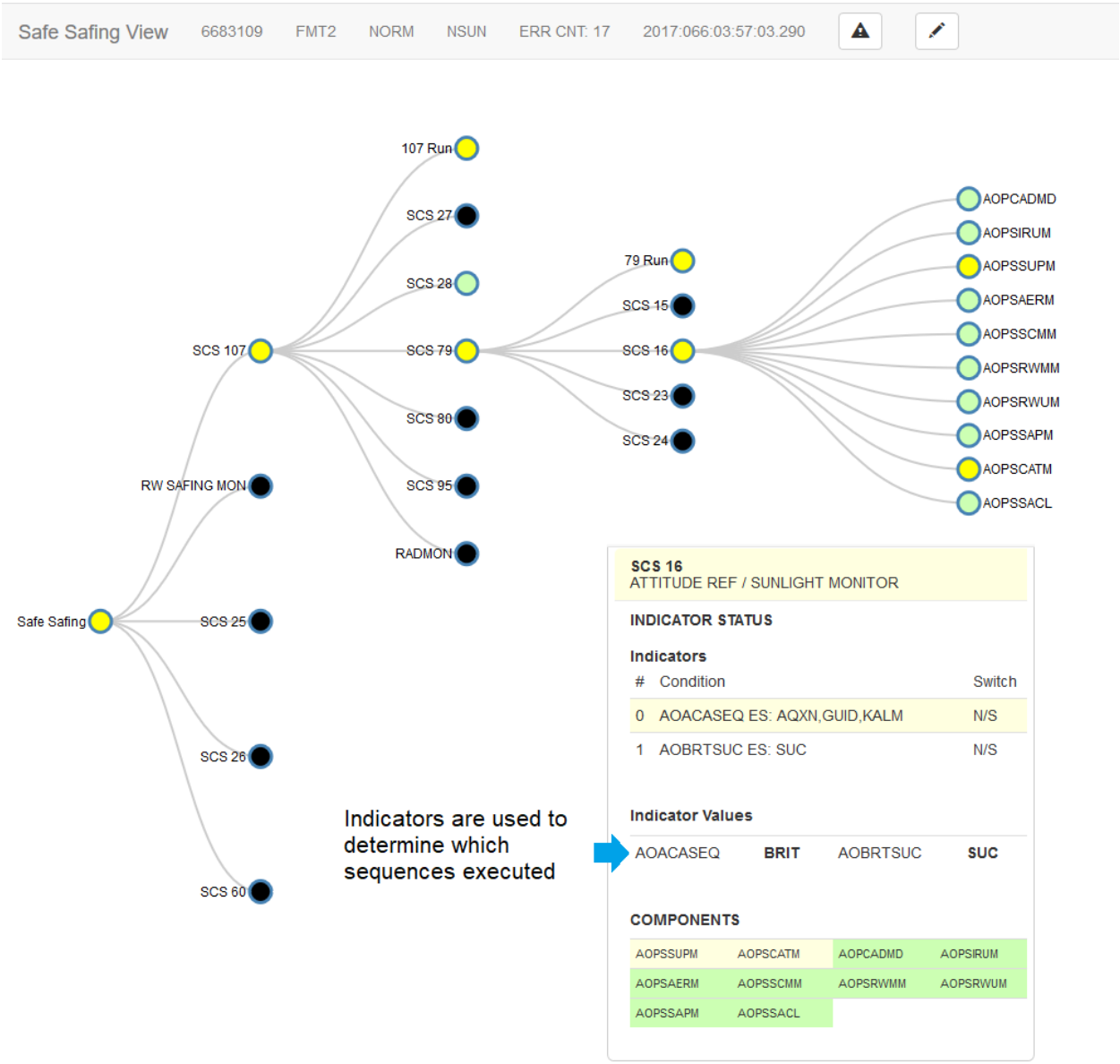

Fig. 4 Telemetry Fed Safing Fault Tree with Safing Action Indicated

Downloaded by 149.154.161.236 on April 12, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2018-2710

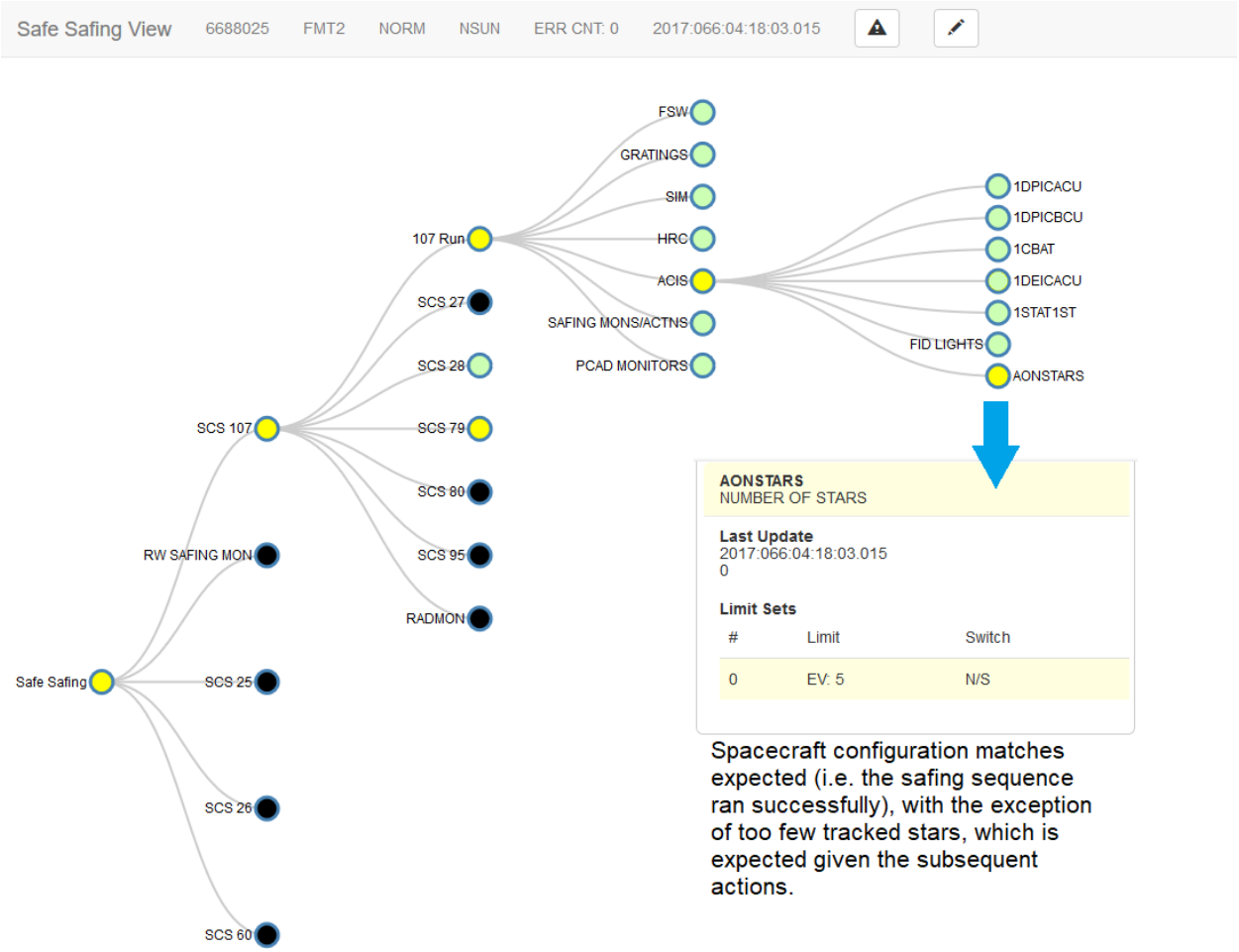

Fig. 5 Verification of Successful Execution of Top Level Safing Action

The real-time fault tree is another single-page web application. The D3.js framework powers the main tree visualization, with additional visual elements drawn from Bootstrap. A configuration file built into the application (JSON) specifies the tree structure, with telemetry points at the leaves, aggregated at each level up to the trunk. Backbone.js provides the foundation for handling user interactions and coordinating changes to the display as data flows to the application. The fault tree format is not limited to a safing subsystem response. It can be used for subsystem-level displays, top-level state of health, or any other telemetry-driven case where a live fault tree will be useful to spacecraft operators and engineers.

IV. Visualization and Analysis of Star Tracker Data

The Chandra Flight Operations Team uses a suite of MATLAB Tools to build mission loads. As part of this process, a set of acquisition and guidance stars is selected for every planned attitude. The selected stars comprise a star catalog, which is uplinked to the spacecraft and used by the flight computer and star tracker for attitude determination and guidance. The tool used for star selection provides options to view and edit star catalogs, giving planners and reviewers an intuitive way to optimize the quality of star catalogs for challenging fields (most fields are adequately handled by automated selection routines). By pulling telemetry into this tool, live star tracker data can be overlaid on the planned star catalogs. In nominal operations this provides operators an intuitive view of control law states and how star data is used. It aids in understanding camera performance and identification of likely causes for dropped stars. In scenarios where Chandra has lost attitude reference, data playbacks allow engineers to watch the sequence of events that led to loss of attitude reference, showing both the big picture of where in the camera field of view each identified bright

object lies and the pixel data for the search boxes. This allows engineers to see the difference between a star, a hot pixel, and a cosmic ray. For near miss cases, it lets them see how far outside the camera search boxes the planned stars landed. Finally, feeding the star data into a pattern-matching algorithm can then enable near-real-time attitude determination, significantly enhancing the ability to recover from loss-of-attitude reference. Figure 6 shows the Simple AXAF User Selected Guide and Acquisition Star Editor (SAUSAGE) with active telemetry overlay.

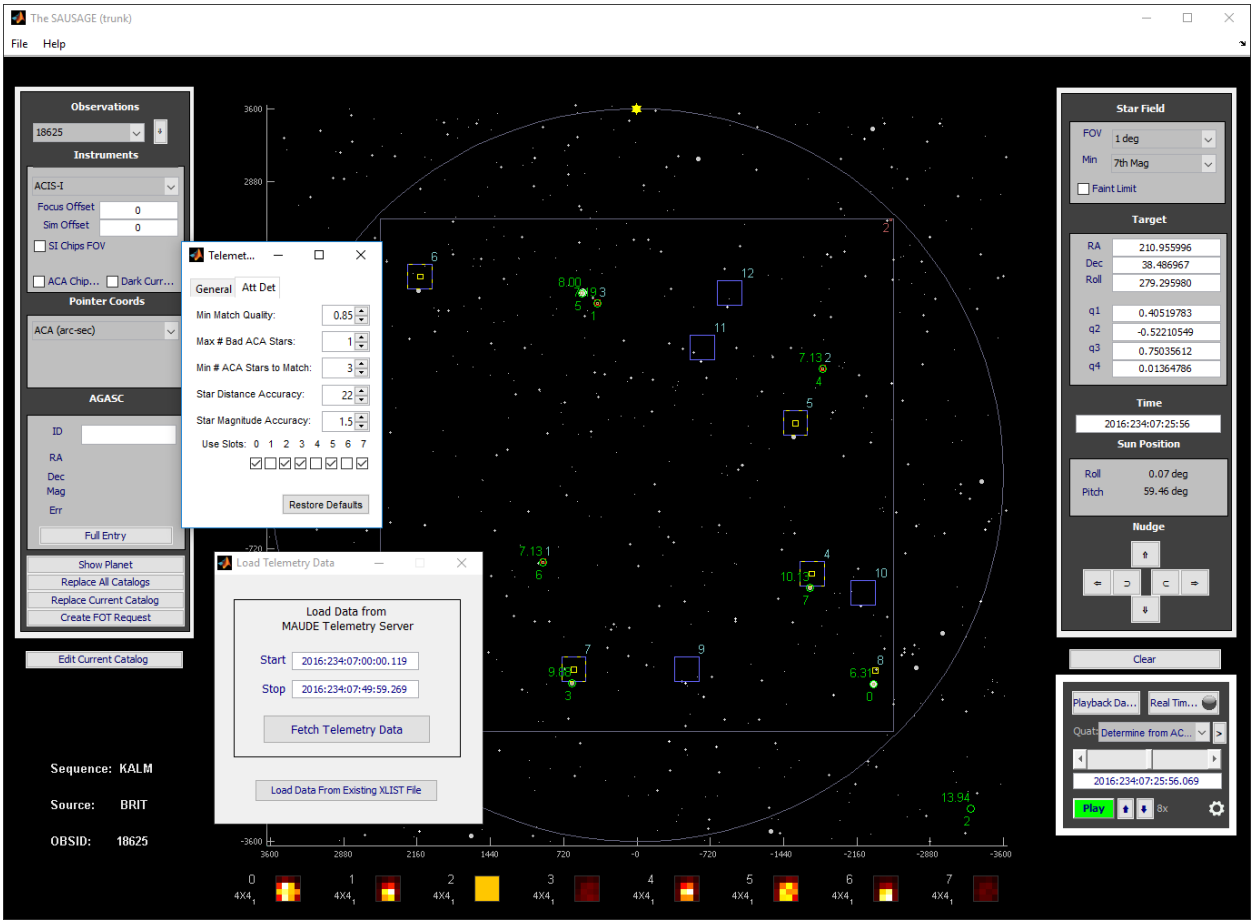

Fig. 6 Telemetry Data in Star Tracking Tool for Attitude Determination

Users can retrieve and overlay historical telemetry data or display real-time telemetry updates through the tool's graphical user interface. The data includes star locations reported by the flight computer based on its processing of star tracker data. More complete data from the star tracking camera can also be displayed to provide additional insight into star tracking status, though this comes at the expense of data retrieval time due to the large number (more than 500) of individual measurements involved (on the order of one second of retrieval time per minute of historical raw star tracking data requested).

Telemetry data is retrieved from the MAUDE server through a flexible and comprehensive class coded in MATLAB. The actual HTTP interface is a combination of a modified version of the built-in urlread MATLAB function and some custom Java classes and functions. The fact that the MAUDE server was intentionally designed to be stateless with a pull-only client model led to the use of MATLAB timers with a separate data retrieval thread written in Java to offload real-time data retrieval processing from the main MATLAB thread to maximize user interface responsiveness.

Downloaded by 149.154.161.236 on April 12, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2018-2710

V. Interactive Trending Plots

Not long after launch, the Chandra Flight Operations team introduced web accessible daily trending plots. Each sub-system engineer defined a set of top-level state-of-health plots that, as a set, provided an overall view of the health of the subsystem. As the engineers start their day, they review the daily plots to check for emerging trends or performance changes that need attention. The daily plots were static web pages, with one plot for each 24-hour period. In 1999, when the mission began, web accessible trending plots were an important innovation that improved the efficiency of the engineering team. As time passed, features such as zooming and panning became commonplace in webpages, but were not yet implemented for the daily plots. The capabilities of MAUDE enabled a significant upgrade to the daily plots. While the daily plots still default to a 24-hour view, by taking advantage of MAUDE's smart subsampling capability[*] users can now zoom out to show days, months, or years. Similarly, users can zoom in to shorter time spans to see full resolution telemetry data. Historical data from the daily plots is seamlessly merged with real-time telemetry on the same display, providing context data for comparison or analysis when responding to a real-time anomaly. A wide-view ticker that identifies features or bi-levels lets users quickly identify and zoom in on areas of interest in the recent past. Since the plots are web accessible, they can be accessed from any device that can support a web browser, providing quick access to top-level real-time telemetry displays with the ability to seamlessly pull in historical data. This greatly enhances the ability of the team to respond to anomalies in real-time, even when they do not have access to the full control center data systems.

Figures 7 and 8 highlight the important features of the interactive trending plots. Engineers can scroll between the plots for their subsystems or scroll in time; these navigation features enable them to efficiently check subsystem health and trends. A secondary time axis with sliders highlights events using bi-level telemetry and provides a simple way to select a time range of interest. Once an event of interest is identified, data traces and rapid zooming allow an immediate detailed look.

The single-page plots web application provides access to a variety of plots through a configuration file (JSON) built into the application. Plotting capabilities are drawn from the Dygraphs package, with additional development work and package tuning undertaken to reach the desired appearance and feature set. Additional front-end elements are provided by Bootstrap and jQueryUI. Backbone.js provides the foundation for handling user interactions and changes to the display in response to user inputs, including coordination of the mixture of live and historical data. We favored a single-page web application over an app for ease of development and use; as implemented, the application runs in almost any web browser on any platform, whereas additional complexity would be introduced with maintaining and supporting an app across multiple platforms.

[*] For a timespan containing more than approximately 100,000 points, the goal of the smart subsets capability is to return a subset of those points that would produce a visually indistinguishable plot given the resolution of the display. The algorithm is based on screen resolution – while one could plot hundreds of thousands of points, there are not enough pixels on the screen to make it possible to distinguish each one. Note that the subsets returned are comprised of real points, not statistics or some other derivative of the data. The plot of the subset enables the user to see the salient features of the dataset – maxima, minima, trends, etc. By progressively zooming in on areas on interest, the user is able to look at all the points (i.e. not a subset) for those areas without the overhead of processing all the points for the whole dataset. Smart subsets was inspired by the viewport resolution aggregation capability presented in Ref. [2].

Downloaded by 149.154.161.236 on April 12, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2018-2710

Fig. 7 Features of Interactive Trending Plots

Downloaded by 149.154.161.236 on April 12, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2018-2710

Downloaded by 149.154.161.236 on April 12, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2018-2710

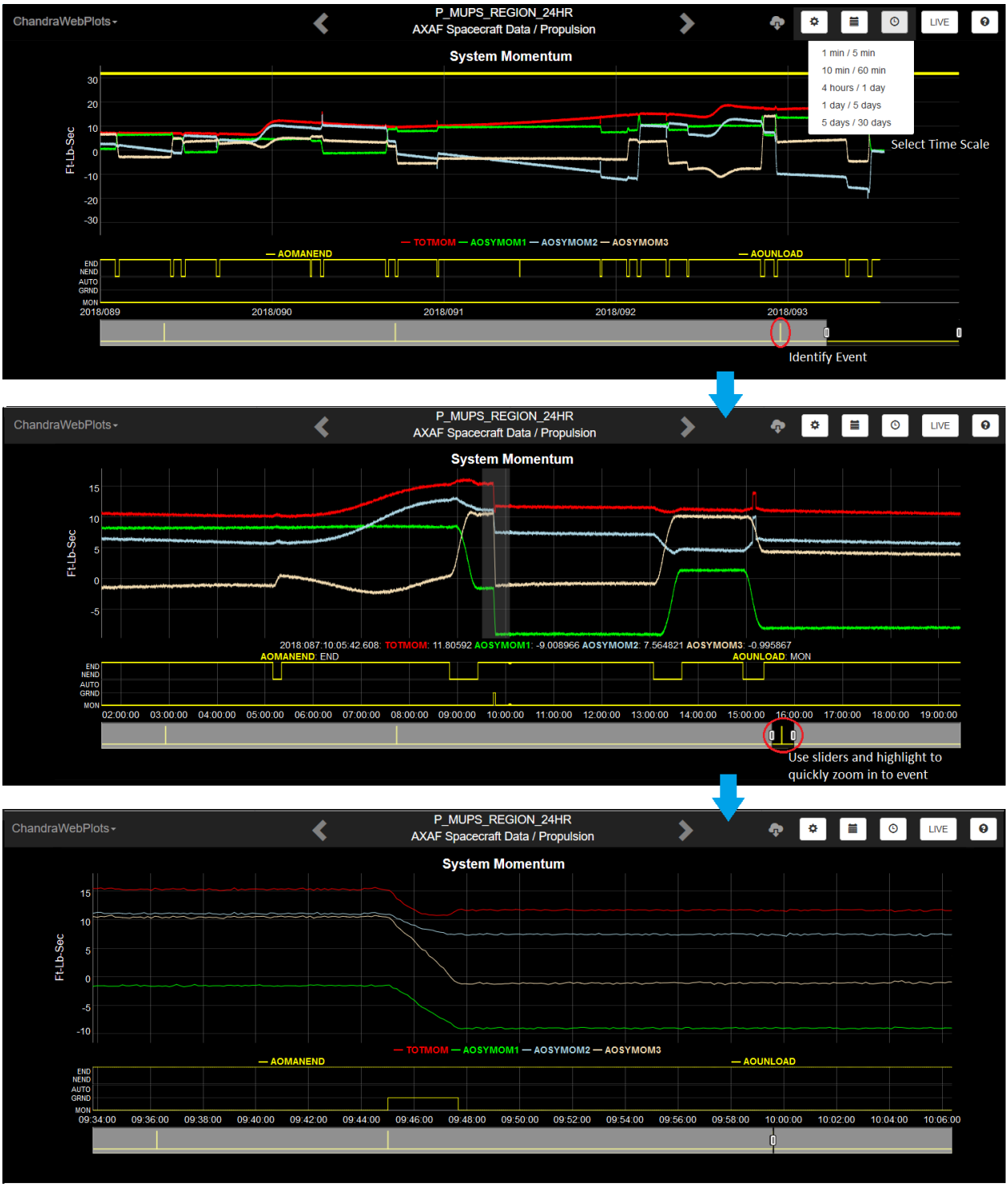

Fig. 8 Rapid Analysis of Important Events

Downloaded by 149.154.161.236 on April 12, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2018-2710

VI. On-the-Fly Telemetry Analysis

When responding to an anomaly, the ability to quickly and easily create a plot of any telemetry point is invaluable. MAUDE provides this capability at the most fundamental level by returning text formatted telemetry data that can be plotted in any spreadsheet tool accepting comma separated values as input. This is a valuable capability, but it requires building the proper query string and going through the steps to open and plot the file in a spreadsheet program. When seconds count, or when searching for correlations among many data sets, a more streamlined approach is desirable. Since the Chandra mission scheduling toolset is built in MATLAB, the flight team has ready access to MATLAB and its built-in plotting features. Further, the team has developed myriad utilities for handling of spacecraft operations-specific tasks. This made MATLAB an obvious choice for creating an on-the-fly telemetry plotting and analysis toolset. The MATLAB telemetry plotter interfaces with the MAUDE server to let the user graphically choose any telemetry measurement for plotting (with search capability that covers measurement descriptions), configure plots interactively, define calculated values on the fly, save and load plot configurations, etc. This tool allows any telemetry data to be plotted without the need for a pre-built configuration file and without needing any specialized knowledge of the data or the tool. It pulls data from MAUDE to display any timeframe of historical data as well as real-time updates, eliminating the need for users to locally store any portion of the telemetry dataset. Figure 9 shows a snapshot of the Telemetry Plotter highlighting some of its interactive features.

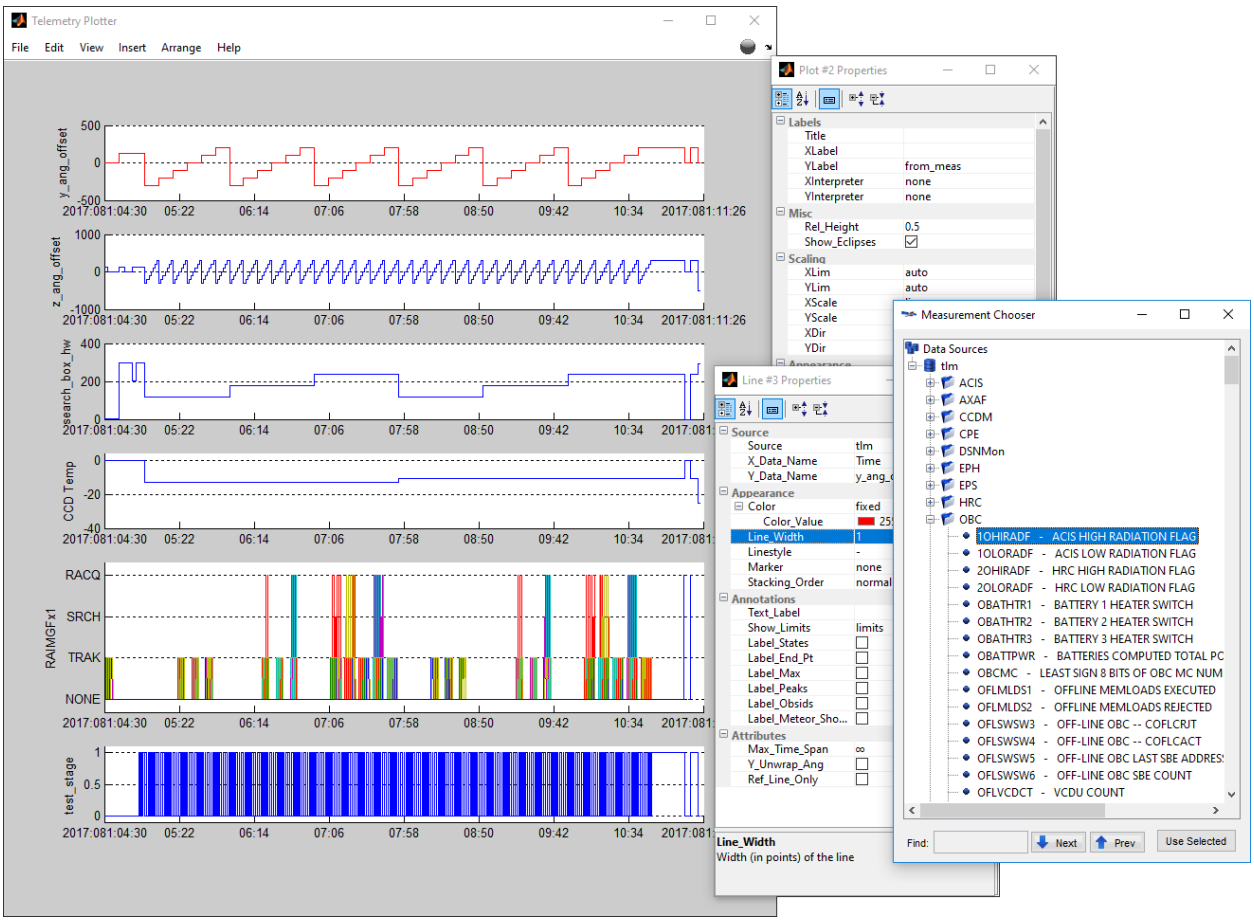

Fig. 9 Interactive On-the-Fly Telemetry Plotter

The same interface can also be used to overlay telemetry data on model outputs used in the mission scheduling process. This capability allows flight controllers to check telemetry against predicted values in real-time and engineers to check spacecraft performance against predicts for both real-time and back-orbit data. It was a critical part of the analysis when a subtle misalignment in the on-board and ground-based calibrations of sensor data allowed a planned

attitude to exceed on-board safing limits. Providing a simple means to both create custom telemetry points and to overlay planned and observed values quickly demonstrated the problem. Figure 10 shows a snapshot of the Maneuver Constraint Checker tool used in mission scheduling overlaid with a custom calculation that converts the raw telemetry data from a sensor using the flight calibration instead of the ground calibration. The custom calculation was easily created within the graphical interface and then overlaid on the planning data. This analysis, which previously would have required multiple data pulls and extensive manipulation, demonstrated the problem that caused the Safe Mode event and allowed to team to rapidly refocus on recovery and return to mission. While this analysis was performed using playback data, observed performance can be overlaid on model predictions in real-time allowing flight controllers to spot diverging trends before any critical thresholds are crossed.

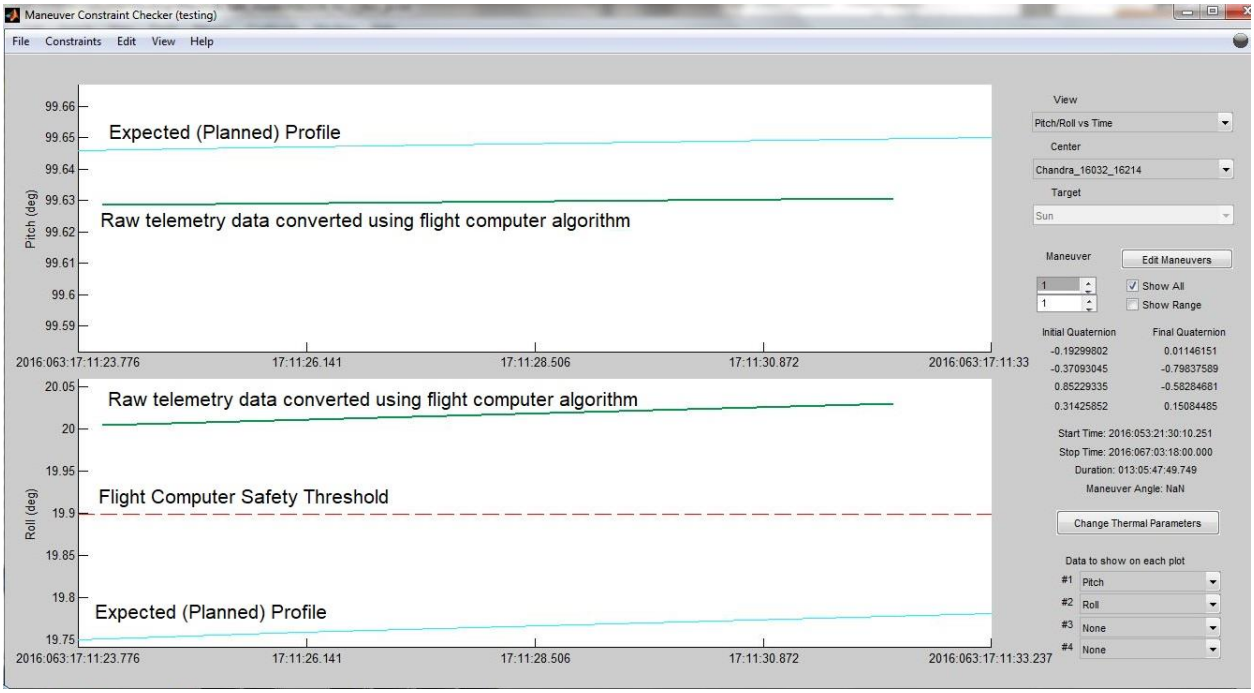

Fig. 10 Comparison of Expected vs Observed Performance Using Telemetry Data

Telemetry data for these tools is retrieved from the MAUDE server through a flexible and comprehensive class coded in MATLAB. This class also provides a convenient API for users from the MATLAB command line or user-generated analysis scripts. The class handles all interaction with the server and queries the user for credentials when required. This provides a portable data analysis and visualization capability with no need for users to store (and keep updated) the telemetry dataset locally.

VII. 3D Spacecraft Model with Thermal Data Overlay

After eighteen years in space, Chandra's thermal protective surfaces have degraded significantly. This degradation has caused several spacecraft systems to exceed their thermal design limits. Mitigations require careful planning of mission schedules to balance heating and cooling of different spacecraft regions. Hundreds of thermistors are limit sensed and closely monitored to verify that the planned attitude profiles are successful in maintaining safe spacecraft temperatures. When a thermistor exceeds its limits, knowing where on the spacecraft the thermistor lies and what its near neighbors are is critical to understanding likely causes and potential mitigations. However, locating thermistors on schematics is a time consuming process. In response to this problem, an interactive, 3D-spacecraft model showing thermistor locations and temperatures, where thermistor color indicates absolute temperature or limit state was developed. The 3-D model provides both an at-a-glance subsystem health check and an intuitive tool to put the data in context when a temperature goes beyond its limits. The model can be run in real-time or from playback data, and provides a sun vector for further context information. Each thermistor can be selected either by clicking on the model or by searching for a mnemonic. When selected, the model shows a description of and the temperature data for that

Downloaded by 149.154.161.236 on April 12, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2018-2710

Downloaded by 149.154.161.236 on April 12, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2018-2710

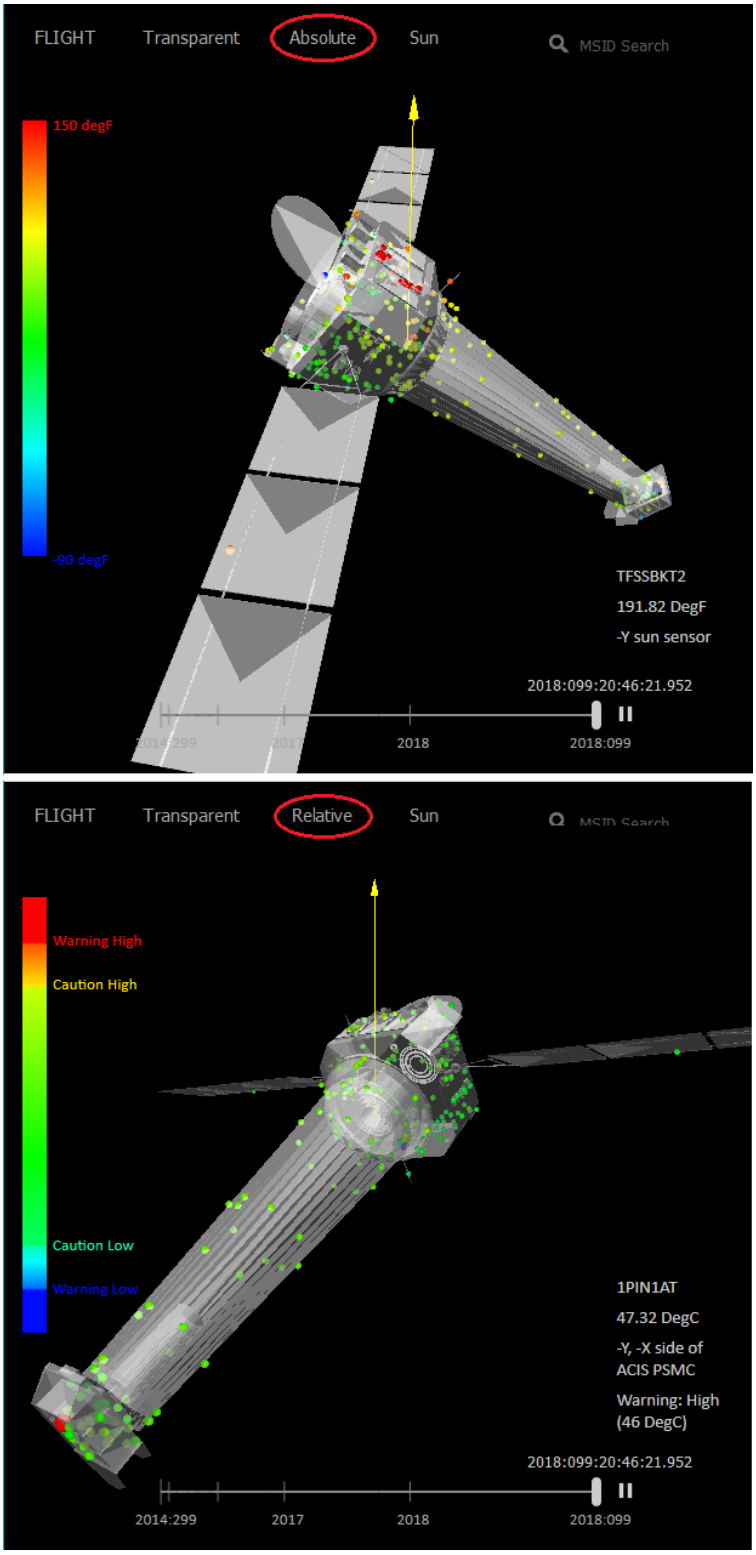

Fig. 11 3-D Thermal Telemetry Viewer

thermistor. Figure 11 shows the model with an absolute temperature scale (top), intuitively showing the cold and hot areas on the spacecraft, and with a relative scale (bottom) that indicates when a thermistor is out of limits. This is particularly useful for a quick check of hundreds of temperatures and rapid contextualization of any out of limits information. Note the single red node on the relative scale (bottom image), which has been highlighted to display the telemetry and database information for that thermistor. In this case, only one thermistor in the area is out of limits and all other adjacent thermistors are at nominal temperatures, indicating a potentially faulty sensor. If several adjacent thermistors were observed to be above or near their limits, environmental heating or a stuck on heater would be suspected as the more likely cause. This type of adjacency information can be time consuming to find in technical drawings, particularly for thermistors monitoring different subsystems (e.g. a panel temperature and a propulsion temperature). The 3D model removes these hurdles and allows operators or engineers to rapidly find nearby thermistors for consistency checks and troubleshooting.

This display was not more technically difficult to create than the other displays highlighted in this paper due to the wide availability of high-level open source web development libraries. With guidance from an experienced software team member, a team of two college interns created the display. One analyzed technical spacecraft drawings to meticulously build the 3-D model and place each thermistor accurately, while the other developed the web page software, including code for interacting with the model and code for processing and displaying telemetry data from the MAUDE server. The model was created in Blender, a widely-used open source 3-D modeling application. Three.js, an open source JavaScript library and API utilizing WebGL was chosen as the framework for rendering the model to simplify code development. An existing in-house JavaScript API for retrieving telemetry data from the MAUDE server was re-used to leverage the server's optional binary data stream to minimize data retrieval time for maximizing user responsiveness.

VIII. SOPs with Integrated Telemetry

Real-time spacecraft Standard Operating Procedures (SOPs) are heavily text based and must be easy to execute in time-pressured situations. For these reasons, Chandra SOPs were developed in Microsoft Word with strict formatting rules. For SOPs where speed of execution is critical (e.g. SOPs that disable safing systems, or switch hardware units), purpose built displays with all of the telemetry needed for SOP execution are developed and maintained. These SOP specific displays are often organized to follow the steps of the SOP, which adds to the maintenance required when updating SOPs. While these displays speed execution, switching attention back-and-forth between the display and SOP telemetry verification tables is still necessary, which adds execution time and risk. Further, development and maintenance of the SOP-driven displays is time-consuming, tedious work. With MAUDE, embedding live telemetry verification tables into full-text Standard Operating Procedures, written in Microsoft Word became possible. Embedded telemetry eliminates inefficient back-and-forth scanning between a procedure and telemetry displays and eliminates the need to develop and maintain displays to accompany critical SOPs.

The original integrated-telemetry SOPs used specially modified editions of the existing Microsoft Word procedures, combined with a backend Visual Basic for Applications (VBA) module facilitating live telemetry data retrieval and processing. As telemetry points listed in the procedure update in the live data stream, the updates are reflected in the Microsoft Word document along with an indication of whether the current state of the telemetry point matches the expected state as listed in the verification table. The second-generation design builds on these capabilities as a single-page web application, with the potential for customization and advanced features beyond easy reach of a Microsoft Word document, such as guiding the operator through the procedure step by step.

Since SOPs are developed and maintained by engineers and flight controllers, not by the software development team, the process for updating the content of an SOP must be simple and user friendly. To this end, a simple plain-text mark-up format was developed and implemented. The format is human readable and easily editable, but designed such that it can be reliably rendered into a consistently formatted web page containing procedure steps, notes, and telemetry verifiers. There are several advantages to this approach: (1) it removes the need for a software developer to handle SOP content; (2) it removes the frustration of applying exacting formatting standards to a Microsoft Word document; (3) it decouples formatting changes from content changes; and (4) SOP content becomes plain text files that are easier to configure and version control than Microsoft Word documents.

Figure 12 shows an example of the SOP mark-up and the rendered SOP. The Observed column populates with real-time telemetry from MAUDE when telemetry flow to the SOP is started. The Observed column is compared to the Expected column for verification and the Observed value is highlighted in green when there is a match. Once the controller checks the Done/Verified box, data to that procedure step is frozen and a circular indicator indicating the verification status at time of completion added. Once a full step is completed, it is marked as such with a completion time-stamp. Once the full SOP is completed a static version of the page can be saved for record keeping. The view shown in Figure 12 reflects an upgrade to existing versions of live telemetry in SOPs; the features regarding step completion are still in development at the time of publication but expected to be completed within the year.

Downloaded by 149.154.161.236 on April 12, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2018-2710

Downloaded by 149.154.161.236 on April 12, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2018-2710

ChandraProc 15289780 FMT2 NORM NPNT ERR CNT: 0 2018:075:00:03:35.842

1. SETUP

2. PREPARE SPACECRAFT FOR RECOVERY FROM BRIGHT STAR HOLD

3. COMPLETE ATTITUDE DETERMINATION

4. IF NECESSARY, MANEUVER AND ACQUIRE DIFFERENT BRIGHT STARS

This step is only executed if the phase and pattern match from the prior step is unsuccessful

Step	Who	What	How	Verifiers	Expected	Observed	Done/Verified
a	OC	Uplink target quaternion update load into Mission SCS slot #216	Uplink AA_Q_BSH_216.cld (if OBC-A Online); Uplink AB_Q_BSH_216.cld (if OBC-B online); See Command Load List for Checksum	ONLMLDS1	INCR 1		☑
b	OC	Prepare to execute the maneuver	Start script A_BSH_MANVR. Enter 216 at the prompt.				☑
c	OC	Command Normal Maneuver Mode	Resume past initial WAIT	AOPCADMD	NMAN	NMAN ●	☑
d	OC	Enable and activate SCS #216	Resume script A_BSH_MANVR	COSCS216S	INAC	INAC ●	☑
e	OC	Start the maneuver	Resume script A_BSH_MANVR; Script uplinks command AOMANUVR	AOFATTMD	MNVR	MNVR ●	☑
f	OC	Confirm maneuver complete	Wait for maneuver execution and then verify telemetry	AOFATTMD	STDY	STDY	☐
g	OC	Command Normal Pointing Mode	Resume script A_BSH_MANVR; Script uplinks command AONPMODE	AOPCADMD	NPNT	NMAN	☐
h	OC	Command Bright Star Hold	Resume script A_BSH_MANVR; Script uplinks command AOBRIGHT	AOACASEQ AOFSTAR	KALM BRIT	ACQ ACQ	☐
i	PCAD	Confirm Successful Acquisition of Bright Stars	Use display EHS_ACA_ALL to confirm acquisition and track of expected stars				☐
If Bright Star Hold was entered autonomously, two OBC errors will be generated - one for the transition to Bright, and one for activating the disabled SCS #28							
j		Return to prior step to perform the phase/pattern match.					☐

5. PERFORM ATTITUDE UPDATE AND EXECUTE (NULL) MANUEVER

Mark-up for Steps 4.h. to 5.0

```
SUBSTEP:
WHO: OC
WHAT: Command Bright Star Hold
HOW:  Resume script A_BSH_MANVR; Script uplinks command AOBRIGHT
VERIFIERS/EXPECTED: AOACASEQ KALM; AOFSTAR BRIT

SUBSTEP:
WHO: PCAD
WHAT: Confirm Successful Acquisition of Bright Stars
HOW:  Use display EHS_ACA_ALL to confirm acquisition and track of expected stars

NOTE: If Bright Star Hold was entered autonomously, two OBC errors will be generated - one for the
transition to Bright, and one for activating the disabled SCS #28

SUBSTEP:
WHAT: Return to prior step to perform the phase/pattern match.

STEP: PERFORM ATTITUDE UPDATE AND EXECUTE (NULL) MANUEVER
```

Fig. 12 Standard Operating Procedure with Embedded Telemetry

Downloaded by 149.154.161.236 on April 12, 2026 | http://arc.aiaa.org | DOI: 10.2514/6.2018-2710

IX. Conclusion

The telemetry clients presented in this paper represent just the beginning of what can be done to improve the way telemetry displays aid operations workflows and situational awareness. Elimination of barriers to telemetry access and the rapid advancement in the field of data visualization have introduced new opportunities for telemetry displays that build understanding of spacecraft functions and intuitively support spacecraft anomaly response. Each of the tools presented contribute to a top-level goal of efficient and intuitive spacecraft operations and troubleshooting of spacecraft anomalies. Chandra Dash and the live safing fault tree can improve response times for out-of-limit scenarios and autonomous spacecraft reconfigurations and help organize information in a way that is most useful to the engineer or operator. Web accessibility and a mobile friendly form factor make these displays particularly useful for engineers supporting anomaly response outside of nominal business hours. The live view of star tracker data allows operators and engineers to see what the star tracker "sees" building intuition and situational awareness. Further, the built-in attitude determination algorithm eliminated a time consuming step in the process of returning to mission after a loss of attitude reference, again improving anomaly response time. Interactive trending plots provide spacecraft engineers a user friendly way to monitor the health of their subsystem and a rapid response tool for top level health and status checks from anywhere. When deeper analysis is required, the Telemetry Plotter provides immediate access to, and capability for analysis of, any telemetry point or set of points. When working with thermal data, the 3-D spacecraft model with telemetry overlay provides critical context information rapidly and with minimal effort. Finally, while still in development, the use of live telemetry in SOPs promises to both streamline on-console operations and reduce level of effort for SOP development and maintenance.

There are a number of common threads amongst the tools explored here, with the most important being a development process based on close collaboration between users and developers. This collaborative approach, which emphasizes a tight feedback loop for innovative features, both prevents resource intensive rework and results in a more usable end product. Where practical, introducing new functionality into familiar tools improves user comfort level, speeds adaptation, and avoids introducing new processes for management, configuration, and control. Platform independence has become an important factor in the way that we work today. This naturally leads development towards single-page web applications, instead of apps, and towards platforms like MATLAB that are supported across multiple operating systems (Linux, Windows, and Mac). Taking advantage of well-vetted and supported open-source software and libraries keeps development costs and resources manageable.

The innovative telemetry displays and applications presented in this paper would have been possible without MAUDE, Chandra's modern telemetry back-end. MAUDE clearly shows that investing the resources to develop a robust back-end that simplifies client development quickly pays dividends once that client development begins. The most direct benefit of investment in the back-end is the elimination of the need for repeated effort and code for client side processing and formatting of data. Just as important, creating a simple user friendly interface empowers end users to become client developers, which expands reach and provides opportunities for staff skill development and engagement. The tools presented here are just a sub-set of the telemetry clients developed since MAUDE was released. Smaller scale projects have been successfully developed by members of the Engineering and Mission Planning teams, enhancing their skill sets and allowing the software development team to focus on other priorities. The best practices, and the concepts and technologies used in the tools presented here, lend themselves to further expansion and customization for missions of every size.

Acknowledgments

The authors would like to thank the Smithsonian Astrophysical Observatory Chandra X-ray Observatory team for creating an environment that encourages and embraces innovation. In particular our Flight Director, Tom Aldcroft, who sets an example for us all with his drive to continuously improve flight operations and bring new technologies and techniques to long standing challenges.

References

[1] Jones, David. "Telemetry from Everywhere with Everything: Building a Modern Telemetry System with Open Source Products" AIAA/SOSTC 21st Annual Improving Space Operations Workshop, 5-6 May 2015, Pasadena, CA. https://info.aiaa.org/tac/SMG/SOSTC/Workshop%20Documents/2015/Track%203%20-%20Exploring%20Space%20Using%20Game%20Changing%20Approaches/5_1345_ImprovingSpaceOps_Jones_final.pdf

[2] Oliveira, H., Lais, A., Francisco, T., Donati, A. "Enabling visualization of large telemetry datasets" Proceedings SpaceOps 2012 Conference, Stockholm, Sweden, 11-15 June 2012.